

Final Project Module 0

Spacial based language

GridFlow

1. Overview:

- Name and purpose of your language.
- Target domain and what problem it solves.

2. Design:

- **Syntax:** Brief rules with examples (variables, control structures, functions).
- **Semantics:** Data types, scoping, and execution model.
- **Features:** Programming paradigms supported and standout features.

3. Sample Program: Include 3 sample programs showcasing language syntax and features.

4. Ecosystem: Potential development tools and library ecosystem.

5. Challenges: Possible implementation challenges and limitations.

""""

GridFlow CLI

Usage: python gridflow.py run <file.gf> [key=value ...]

python gridflow.py check <file.gf>

python gridflow.py demo

""""

import sys

from lexer import lex

from parser import Parser, GridFlowTypeError, GridFlowParseError

from interpreter import Interpreter, GridFlowRuntimeError

— Runner

```

def run_file(path: str, inputs: dict, verbose: bool = False):
    with open(path) as f:
        source = f.read()
    run_source(source, inputs, verbose=verbose, label=path)

def run_source(source: str, inputs: dict, verbose: bool = False, label: str = "<source>"):
    print(f"\n{'—'*50}")
    print(f"  GridFlow — {label}")
    print(f"{'—'*50}")

    # Lex
    tokens = lex(source)
    if verbose:
        print(f"  Lexer produced {len(tokens)} tokens")

    # Parse
    parser = Parser(tokens)
    try:
        graph = parser.parse()
    except (GridFlowTypeError, GridFlowParseError) as e:
        print(f"  ERROR: {e}")
        return

    print(f"  Graph: {len(graph.cells)} cells, {len(graph.wires)} wires")
    print(f"  Inputs: {[c.port_name for c in graph.input_ports]}")
    print(f"  Outputs: {[c.port_name for c in graph.output_ports]}")

    # Run
    interp = Interpreter(graph)

```

```

try:
    outputs = interp.run(inputs, verbose=verbose)
except GridFlowRuntimeError as e:
    print(f" RUNTIME ERROR: {e}")
    return

```

```

print(f"\n Results:")
for k, v in outputs.items():
    print(f"   {k} = {v!r}")
print()

```

— Sample programs

```

SAMPLE_1 = """
name:scalar >—[trim]—[upper]—< result:scalar
"""

```

```

SAMPLE_2 = """
score:scalar >—[≥60]—< grade:scalar
"""

```

```

SAMPLE_3 = """
a:scalar >—————|
                {collect}—(avg)—< average:scalar
b:scalar >—————|
"""

```

```

SAMPLE_4_PIPELINE = """

```

plate shout:

```
text:scalar >—[trim]—[upper]—[!]  
—< out:scalar  
end
```

```
message:scalar >—[shout]—< result:scalar  
""
```

— CLI

```
def parse_kv_args(args):  
    inputs = {}  
    for arg in args:  
        if "=" in arg:  
            k, v = arg.split("=", 1)  
            # Try to coerce to number  
            try: inputs[k] = int(v)  
            except ValueError:  
                try: inputs[k] = float(v)  
                except ValueError:  
                    inputs[k] = v  
    return inputs
```

```
def cmd_demo():  
    print("\n" + "="*50)  
    print("  GridFlow — Demo Programs")  
    print("="*50)  
  
    run_source(  

```

```
SAMPLE_1,  
{"name": " hello world "},  
label="Sample 1: string pipeline"  
)
```

```
run_source(  
    SAMPLE_2,  
    {"score": 75},  
    label="Sample 2: gate / branch"  
)
```

```
run_source(  
    SAMPLE_3,  
    {"a": 40, "b": 60},  
    label="Sample 3: merge and reduce (average)"  
)
```

```
def cmd_run(args):  
    if not args:  
        print("Usage: python gridflow.py run <file.gf> [key=value ...]")  
        sys.exit(1)  
    path = args[0]  
    inputs = parse_kv_args(args[1:])  
    run_file(path, inputs, verbose="--verbose" in args)
```

```
def cmd_check(args):  
    if not args:  
        print("Usage: python gridflow.py check <file.gf>")  
        sys.exit(1)
```

```

path = args[0]
with open(path) as f:
    source = f.read()
tokens = lex(source)
parser = Parser(tokens)
try:
    graph = parser.parse()
    print(f"OK — {len(graph.cells)} cells, {len(graph.wires)} wires, "
          f"{len(graph.plates)} plates")
except (GridFlowTypeError, GridFlowParseError) as e:
    print(f"TYPE ERROR: {e}")
    sys.exit(1)

if __name__ == "__main__":
    args = sys.argv[1:]
    if not args or args[0] == "demo":
        cmd_demo()
    elif args[0] == "run":
        cmd_run(args[1:])
    elif args[0] == "check":
        cmd_check(args[1:])
    else:
        print("Commands: run, check, demo")
        sys.exit(1)

```

File Format: Using .gf files, the parser figures out cell width/alignment (Freeform ASCII)
 Type System: Statically typed - wire shapes declared upfront (scalar, stream, signal, bag)
 Plates (functions): Defined at top, used below
 Loops: Iteration works via feedback wire + [@] latch model

Code is a 2D grid. Data flows directionally like a circuit.

→ [input] → [parse]



[validate] →

Here's the full design for GridFlow — a spatial, 2D programming language where the source code is the dataflow diagram. You write on a grid, and data moves across it like electricity through a circuit.

Core philosophy

In GridFlow, the program has no linear reading order. A cell's position is its logic. Data enters from the edges, flows through transformation cells, branches diagonally, merges, splits, and exits. There is no call stack — only a signal propagating across a plane.

The cell vocabulary

Every cell is a single character (or short glyph). The grid is your canvas.

Routing cells — direct data movement

GlyphNameBehaviour → ← ↑ ↓ Wire Passes data unchanged in that direction + Junction Passes data in all connected directions + Crossover Two wires cross without interacting · Tap Passes data through AND emits a copy downward

Transform cells — modify the value

GlyphNameBehaviour [] Box Named function — label inside, e.g. [+1], [×2], [trim]{

} Accumulator Collects flowing values into a list() Reducer Collapses collected values — e.g. (sum), (max), (join) ~ Delay Buffers one tick before passing on

Control cells — branch the flow

GlyphNameBehaviour ? Gate Passes data right if truthy, down if falsy ! Invert Flips boolean values = Compare Emits true/false rightward, consuming two inputs # Counter Emits the count of values that have passed through

I/O cells — edges of the program

GlyphNameBehaviour > Input port Data enters the grid here (named on left margin) < Output port Data exits the grid here (named on right margin) * Constant Emits a literal value on every tick

Reading a GridFlow program

The grid has a coordinate system — columns are X, rows are Y. Programs are written left-to-right and top-to-bottom by convention, but data can flow any direction. The runtime traces signal paths from all > input ports simultaneously.

Example 1 — a simple pipeline

name > ————— [trim] ————— [upper] ————— < result

One input, two transforms in sequence, one output. This is identical to result = upper(trim(name)) in a conventional language.

Example 2 — a branch

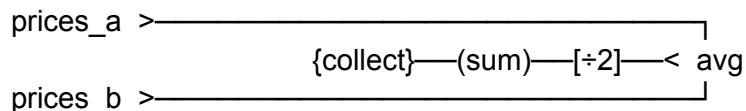
score > ————— [≥60] — ? ————— < grade_pass



[×0] ————— < grade_fail

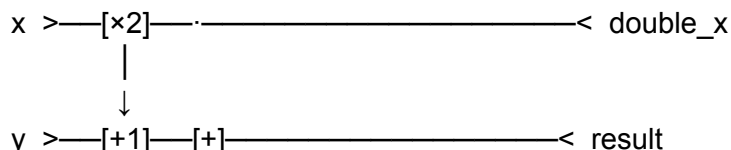
The ? gate sends the value right if the score passes the threshold, down if it fails. Two separate output streams.

Example 3 — a merge and reduce



Two input streams feed into an accumulator {collect}, which drains into a (sum) reducer, then a divider — computing the average. Data from both streams merges at the {collect} cell.

Example 4 — a 2D computation (the real power)



The · tap on x's wire sends a copy downward into the addition cell, where it combines with y+1. This is result = (x*2) + (y+1) — expressed spatially, with the data dependency literally visible as a vertical line.

The type system

GridFlow is typed by shape, not by declaration. Every wire carries one of these shapes, inferred from what's connected to it:

scalar — a single value (number, string, bool)

stream — a sequence of values flowing over time

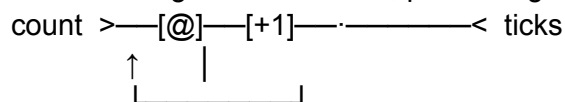
signal — a boolean pulse (used for gates and control)

bag — an unordered collection (output of accumulators)

Type mismatches are spatial — the compiler draws a red **X** on the grid cell where an incompatible connection is made. You see the type error at the location of the problem.

Loops and recursion

There are no for loops in the traditional sense. Instead, GridFlow has feedback wires — a wire that routes back leftward on the grid to an earlier cell. A small @ marker on the wire signals a feedback edge to the runtime, preventing infinite evaluation.



The [<@] cell is a latch — it holds the value, emits it, then accepts the updated value on the next tick. This is how all iteration works.

Modules and reuse

A rectangular region of the grid can be named and reused as a plate — GridFlow's equivalent of a function:

plate: normalize

input >---[<min]---[<range]---< output

Plates are called by placing a [normalize] box anywhere in the main grid. Internally it expands to the plate's layout. Plates can be nested.

Concurrency — it's free

Because GridFlow has no shared mutable state, every cell that has all its inputs ready executes in parallel automatically. The runtime is a dataflow engine — it fires cells as soon as their input wires are live. Concurrency isn't a feature you opt into; it's the default execution model of the grid.

Here's an interactive playground where you can see GridFlow programs animate their data flow in real time:

```
::view-transition-group(*),  
::view-transition-old(*),  
::view-transition-new(*) {  
  animation-duration: 0.25s;  
  animation-timing-function: cubic-bezier(0.19, 1, 0.22, 1);  
}
```

VvisualizeVvisualize show_widgetStep through each program or hit Run to watch signals travel cell by cell across the grid. Switch between all four programs to see the different flow patterns.

What makes GridFlow genuinely novel

Most dataflow languages (LabVIEW, Pure Data, Node-RED) still use a node-and-wire diagram — a GUI you click around in. GridFlow is different: the text file is the grid. You write ASCII art and it executes. A few implications:

Spatial locality is semantic. Two cells next to each other are directly connected. Distance on the grid corresponds to distance in the computation. You can see how far data travels.

Debugging is spatial. Instead of a stack trace, the debugger highlights which cells are currently carrying a live signal. The "error location" is literally a place on the grid you can point at.

Diffing is visual. A git diff of a GridFlow program shows exactly which cells changed — and because position is meaning, you can read what the change does just by looking at where it is.

Parallelism is topological. The compiler finds all cells with no shared wires and schedules them concurrently without any annotation from the programmer. The grid's structure is the concurrency specification.